

GestioneViaggi — Overview per Agente AI

Versione: 1.0
Data: 2026-06-27
Scopo: Documento di riferimento rapido per agenti AI che lavorano su questo progetto.

1. COSA FA IL PROGETTO

GestioneViaggi è un gestionale desktop (MAUI + Blazor) multi-tenant per tour operator offroad (primo cliente già in effettivo: Sardegna Fuori Traccia). Gestisce:

- **Anagrafica clienti** (con storico viaggi, documenti, contatti)
- **Viaggi e date di partenza** (tipologia, alloggi, partecipanti)
- **Contabilità** (movimenti, fatture attive, scadenziario, registro IVA, cash flow)
- **Stampe PDF** (bilancio viaggio, rooming list, fatture, scadenziario)
- **Export Excel** (clienti, movimenti, ecc.)
- **Email** ai partecipanti (via SMTP aziendale o Resend API)
- **Multi-azienda (multi-tenant)** con ruolo SuperAdmin che vede tutte le aziende

L'architettura è **DB-First**: tutta la logica SQL vive in function PostgreSQL. Zero SQL inline nel codice C#.

2. STACK TECNOLOGICO

Layer	Tecnologia
App framework	.NET 9 MAUI + Blazor (WebView)
UI Component Library	MudBlazor
Database	PostgreSQL 17.5
DB Driver	Npgsql (nativo, AOT-ready)
PDF Generation	QuestPDF (Community license)
Excel Export	ClosedXML
Email	MailKit (SMTP) + Resend API (fallback)
Rich Text Editor	Quill.js (via BlazoredTextEditor)
Auth	Blazor AuthStateProvider custom
Target Platforms	macOS Catalyst, Windows (primari)

3. REGOLE MANDATORY — LEGGERE PRIMA DI TUTTO

3.1 DB-FIRST (ASSOLUTA)

Nessun SQL nel codice C#. Ogni query deve essere una function PostgreSQL.

Pattern obbligatorio:

```
// SBAGLIATO – SQL inline
var result = await db.QueryAsync<Foo>("SELECT * FROM foo WHERE id = @id",
new { id });

// CORRETTO – chiamata a function DB
var result = await db.QueryAsync<Foo>("SELECT * FROM
fn_get_foo(@p_id::INTEGER)", new { p_id = id });
```

Dopo ogni function DB creata o modificata → **aggiornare Documents/Funzioni_DB.md.**

Script SQL deploy locale:

```
docker exec -i postgres_db psql -U postgres -d gestione_viaggi <
SqlScripts/NNN_NomeScript.sql
```

3.2 COMPONENTI SHARED (ASSOLUTA)

Usare sempre i componenti da **Components/Shared/**. Non duplicare logica.

Dopo ogni componente creato o modificato → **aggiornare Documents/ComponentiShared.md.**

Documento di riferimento completo: **Documents/ComponentiShared.md.**

3.3 UI — FORM DI EDIT

1. **SetFocus sul primo campo:** sempre, all'apertura della form.
2. **Maiuscolo forzato:** tutti i campi alfanumerici → `Style="text-transform:uppercase"` + normalizzazione nel model. Eccezione: campi destinati a pagine web.
3. **Tabulazione con JS helper:** usare il `dialogFormHelper.js` già presente per gestire il tab tra campi. È mandatorio in tutte le form di edit.
4. **BackdropClick=false:** tutte le modali devono avere `new DialogOptions { BackdropClick = false }` per impedire chiusure accidentali.

3.4 VALIDAZIONE

Il sistema di validazione è centralizzato in **Validation/**. Non scrivere regole inline nei componenti.

Usare/estendere i validator già presenti. Vedi **Documents/Gestione_check.md** per il catalogo completo.

4. STRUTTURA CARTELLE

```
GestioneViaggi/
├─ MauiProgram.cs
```

→ DI container, registrazione tutti i

```

servizi
├── App.xaml / App.xaml.cs      → Entry point MAUI, window sizing
(1200x800)
├── appsettings.json            → Connection string PRODUZIONE (Supabase)
├── appsettings.Development.json → Connection string SVILUPPO (Docker
locale)
├── Components/                → Tutti i componenti Razor (167 file)
│   ├── Layout/                → MainLayout, DashboardLayout, LoginLayout
│   ├── Pages/                 → Pagine applicative
│   └── Shared/                 → Componenti riutilizzabili (libreria
interna)
├── Models/                    → Entità, DTO, enum (~90 file)
│   ├── DTOs/                  → Data Transfer Objects per stampe/query
complesse
│   ├── UI/                    → Modelli per componenti UI
(ColumnMetadata)
│   └── Exceptions/            → Eccezioni custom
├── Services/                  → Business logic (90+ file)
│   ├── Database/              → Connessione e query PostgreSQL
│   ├── Authentication/        → Login, sessione, password reset
│   ├── CRUD/                  → Un servizio per ogni entità
│   ├── Email/                 → SMTP + Resend, template HTML
│   ├── Printing/              → QuestPDF: un printer per ogni report
│   └── Export/                 → Excel (ClosedXML), XML fattura
elettronica
├── Session/                   → SessionManager, TenantContext (multi-
tenant)
├── Shared/                     → FileOpener, ExchangeRate,
BrowserLauncher
├── Navigation/                → TabManagerService
├── UI/                         → StatusBar, DataGridHelper
├── ExternalApis/              → CurrencyApiService (frankfurter.app)
└── Tools/                     → DatabaseDocumentationService

├── Repositories/              → Solo ClienteRepository (pattern non
generalizzato)
├── Validation/                → Sistema validazione centralizzato
│   ├── Core/                  → IValidator, ValidationResult,
ValidationMessages
│   ├── Syntax/                → Formato (P.IVA, CF, email, telefono,
IBAN)
│   ├── Semantic/              → Logica (date, numeri, geografia)
│   └── Business/              → Regole di dominio (possono accedere al
DB)
├── Statistics/                → Classi statistiche per dashboard
├── Helpers/                   → DatabaseExceptionHandler,
EntityNormalizer
├── Migrazione_Dati_Oracle/     → Import da Oracle (usato una tantum)
└── SqlScripts/                → 350+ script SQL, numerati

```

```

progressivamente
├── wwwroot/
│   ├── index.html
│   ├── css/
│   └── js/
├── login.js
│   └── lib/
├── Documents/
├── SqlScripts/
├── Resources/
└── Platforms/
    (Mac/Win/iOS/Android)

```

- Asset statici web
- Entry HTML Blazor
- premium-saas-theme.css (tema MudBlazor)
- dialogFormHelper.js, focusHelper.js,
- Bootstrap, Quill.js
- Documentazione progetto (30+ file MD)
- Script SQL numerati sequenzialmente
- Font (Lato, OpenSans), icone, splash
- Configurazioni platform-specific

5. DATABASE — CONNESSIONI

Sviluppo (Docker locale, `appsettings.Development.json`)

```

Host=127.0.0.1
Port=5432
Database=gestione_viaggi
Username=postgres
Password=postgres
Pool: MinPoolSize=1, MaxPoolSize=20
Container: postgres_db (PostgreSQL 17.5)

```

Deploy script locale:

```

docker exec -i postgres_db psql -U postgres -d gestione_viaggi <
SqlScripts/NNN_Script.sql

```

Produzione (Supabase, `appsettings.json`)

```

Server=aws-1-eu-central-1.pooler.supabase.com
Port=6543 (PgBouncer transaction mode)
Database=postgres
Pool: MinPoolSize=0, MaxPoolSize=30
SSL: Require

```

PgBouncer in transaction mode significa: **nessuna prepared statement, nessuna sessione persistente**. Ogni query è autonoma.

Pool Configuration (memoria)

- Docker dev: MaxPoolSize=20, MinPoolSize=1, IdleLifetime=300s
- Supabase prod: MaxPoolSize=10, MinPoolSize=0, IdleLifetime=180s, ConnectionLifetime=600s
- **ConnectionPruningInterval=10s** per cleanup connessioni idle

Nota Dapper + PostgreSQL

```
// Sempre usare cast espliciti per evitare errore 42883 (overload resolution)
new { p_data = myDate?.Date as object ?? DBNull.Value } // DateTime? → ::DATE
// Oppure nei parametri SQL: @p_data::DATE, @p_nome::VARCHAR
```

6. ARCHITETTURA SERVIZI

Registrazione DI (**MauiProgram.cs**)

Lifetime	Usato per
Singleton	IDatabaseService , IDatabaseConnectionManager , SessionManager , Print services (thread-safe)
Scoped	Tutti i CRUD services, auth, navigation, UI services
Transient	Import services Oracle/Excel, MudLocalizer

Pattern CRUD Service

Ogni entità ha un servizio **XxxService.cs** che:

1. Inietta **IDatabaseService** e **ILogger**
2. Chiama sempre **function PostgreSQL** (mai SQL inline)
3. Usa Npgsql con **DefaultTypeMap.MatchNamesWithUnderscores = true** per mapping snake_case → PascalCase

Multi-Tenant (**TenantContext**)

- **ITenantContext** (Scoped) espone **AziendaId** dell'utente corrente
- SuperAdmin vede tutte le aziende → usa **AziendaSelect** per filtrare
- **ISessionManager** (Singleton) gestisce la sessione utente persistente

7. COMPONENTI UI — MAPPA RAPIDA

Documento completo: **Documents/ComponentiShared.md**

Grid

- **EnterpriseDataGrid** → **MudDataGrid** con toolbar (titolo + search + azioni), sempre usarlo
- **EnterpriseActionsColumn** → colonna azioni (modifica/elimina) standard

- **EnterprisePager** → paginazione italiana

Select / Autocomplete (tutti in **Components/Shared/**)

Componente	Sorgente dati
AziendaSelect	ana_aziende
ClienteSelect	ana_clienti
FornitoreSelect	ana_fornitori (filtrato per azienda)
ControparteSelect	ana_controparti (filtro ATTIVO/PASSIVO)
ViaggioSelect	ana_viaggi
ViaggioMultiSelect	ana_viaggi (selezione multipla)
ValutaSelect	ana_valute
CausaleSelect	ana_tipi_causali
AliquotaIvaSelect	ana_aliquote_iva
CountrySelect	eba_countries
RegioneSelect	ana_geo_regioni_ita
ProvinciaSelect	ana_geo_province_ita
ComuneSelect	ana_geo_comuni
RuoloSelect	IRoleService
CicloSelect	Valori statici (ATTIVO/PASSIVO)
UrgenzaSelect	Valori statici (SCADUTO/URGENTE/IN_SCADENZA/NORMALE)

Dialog condivisi

- **SendEmailDialog** → Email a partecipanti viaggio (Trip Mode) o diretta (Direct Mode)
- **StampaMovimentiDialog** → Filtri stampa movimenti contabili
- **StampaScadenzarioDialog** → Filtri stampa scadenziario
- **SelezioneBancaDialog** → Selezione conto bancario per stampa fattura
- **DeleteConfirmationDialog** → Conferma eliminazione standard

Export

- **ExcelExportButton** → bottone standard export Excel con spinner

8. STAMPE PDF

Libreria: **QuestPDF** (Community).

Ogni report ha la coppia **XxxPrintService** (recupera dati DB) + **XxxPrinter** (genera PDF).

Report	Service	Printer
Scheda viaggio	TravelPrintService	ViaggiPrinter
Rooming list	RoomingListPrintService	RoomingListPrinter
Bilancio viaggio	BilancioViaggioPrintService	—
Movimenti contabili	MovTransazioniPrintService	MovTransazioniPrinter
Registro IVA	RegistroIvaPrintService	RegistroIvaPrinter
Scadenzario	ScadenzarioPrintService	ScadenzarioPrinter
Fattura attiva	FatturaAttivaPrintService	FatturaAttivaPrinter

Helper condivisi:

- `ReportHeaderHelper.ComposeCompanyHeader()` → intestazione standard con logo azienda
- `ReportHeaderHelper.ComposeFooter()` → footer con numerazione pagine
- `ReportHeaderHelper.BrandColors` → colori brand condivisi
- `PdfOpenerService.OpenPdfAsync()` → chiede conferma e apre il PDF
- `FileOpenerService.OpenFileAsync()` → generico per qualsiasi file

9. EMAIL

Factory pattern: `EmailSenderFactory` sceglie automaticamente tra:

- `SmtEmailSender` → configurazione SMTP dell'azienda (da `ana_aziende_smtp`)
- `ResendEmailSender` → fallback via Resend API

Template HTML: `CompanyEmailTemplate.GetHtmlBody()` → layout responsive 600px con logo aziendale.

10. AUTENTICAZIONE E SESSIONE

- Login: `AuthenticationService` → chiama function DB → verifica credenziali
- Stato auth: `CustomAuthStateProvider` → Blazor `AuthenticationStateProvider`
- Sessione: `SessionManager` (Singleton) → persiste in `FileStorageProvider` tra restart
- Tenant: `TenantContext` (Scoped) → espone `AziendaId` corrente
- Ruoli: `SuperAdmin`, `Admin`, `User`

11. VALIDAZIONE

Sistema in `Validation/`. Tre livelli:

1. **Syntax** (stateless, < 1ms) → formato P.IVA, CF, email, telefono, IBAN, codici
2. **Semantic** (stateless, < 5ms) → range date, numeri positivi, CAP
3. **Business** (può accedere DB) → unicità, FK, regole di dominio

Pattern risultato:

```
public class ValidationResult {  
    public bool IsValid { get; set; }  
    public string ErrorMessage { get; set; } // in italiano  
    public string ErrorCode { get; set; }    // es. "CHK_PIVA_001"  
}
```

Errori DB → `DbErrorTranslator.Translate(PostgresException)` → messaggio italiano user-friendly.

12. STATISTICHE DASHBOARD

Classi in `Statistics/`:

- `StatisticCountAziende` → numero aziende
 - `StatisticCountClienti` → numero clienti
 - `StatisticCountViaggi` → totale viaggi
 - `StatisticCountViaggiFatti` → viaggi completati
 - `StatisticCountViaggiDaFare` → viaggi futuri
 - `StatisticRevenue` → ricavi
 - `StatisticYearService` → filtro per anno
-

13. PAGINE PRINCIPALI (routing)

Pagina	Percorso	Scopo
Login	<code>/login</code>	Autenticazione
Dashboard Admin	<code>/dashboard</code>	KPI aziendali
Dashboard SuperAdmin	<code>/superadmin</code>	Vista globale
Clienti	<code>/clienti</code>	Anagrafica clienti
Viaggi	<code>/viaggi</code>	Gestione viaggi
Movimenti	<code>/movimenti</code>	Transazioni contabili
Fatture Attive	<code>/fatture</code>	Stampa fatture
Aziende	<code>/aziende</code>	Anagrafica aziende (SuperAdmin)
Utenti	<code>/utenti</code>	Gestione utenti
Tabelle	<code>/tabelle/*</code>	Lookup tables
Strumenti	<code>/strumenti/*</code>	Import, documentazione DB

14. JS HELPERS (`wwwroot/js/`)

File	Scopo
<code>dialogFormHelper.js</code>	Gestione tabulazione campi nelle form (MANDATORIO nelle form edit)
<code>focusHelper.js</code>	SetFocus programmatico su elementi Blazor
<code>themeHelper.js</code>	Switching light/dark theme
<code>login.js</code>	Logica pagina login
<code>utils.js</code>	Utility generiche

15. DOCUMENTAZIONE INTERNA (cartella `Documents/`)

File	Contenuto
<code>Funzioni_DB.md</code>	Single source of truth per tutte le function PostgreSQL. Aggiornare SEMPRE dopo ogni modifica DB.
<code>ComponentiShared.md</code>	Single source of truth per tutti i componenti shared. Aggiornare SEMPRE dopo ogni modifica.
<code>Gestione_check.md</code>	Architettura validazione, catalogo validatori, DbErrorTranslator
<code>DataBaseLocale.md</code>	Credenziali e comandi Docker per sviluppo locale
<code>Multi_Tenancy_Architecture.md</code>	Design multi-tenant
<code>CRUD_PATTERN.md</code>	Pattern standard per CRUD services
<code>PDF_Creation_Standard.md</code>	Standard generazione PDF con QuestPDF
<code>Enterprise_DataGrid.md</code>	Documentazione EnterpriseDataGrid
<code>Analisi_Preliminare_Sito_Web_SFT.md</code>	Analisi progetto nuovo sito SFT (2026-06-19)

16. SCRIPT SQL — CONVENZIONE NUMERAZIONE

Gli script in `SqlScripts/` sono numerati progressivamente (es. `001_`, `002_`, ...). L'ultimo numero usato determina il prossimo da assegnare.

Naming convention:

```
NNN_TipoOperazione_NomeEntita.sql
es: 401_Create_FnGetClientiAttivi.sql
```

17. FLUSSO TIPICO AGGIUNTA FEATURE

1. **Creare function PostgreSQL** in un file `NNN_*.sql`

2. **Deploy su Docker** con `docker exec -i ...`
 3. **Aggiornare Documents/Funzioni_DB.md**
 4. **Creare/aggiornare Model** in `Models/`
 5. **Creare/aggiornare Service** in `Services/CRUD/` — chiama la function, niente SQL inline
 6. **Registrare il service** in `MauiParam.cs` se nuovo
 7. **Creare componente Shared** se si tratta di un elemento riutilizzabile (select, dialog, ecc.)
 8. **Aggiornare Documents/ComponentiShared.md** se creato/modificato un componente
 9. **Creare/aggiornare la pagina** in `Components/Pages/`
 10. **Rispettare UI rules:** SetFocus primo campo, uppercase, dialogFormHelper.js
-

18. NOTE SUPABASE / PRODUZIONE

- Il DB in produzione è su **Supabase** (PostgreSQL managiato AWS eu-central-1)
- La connessione passa per **PgBouncer** (porta 6543, transaction mode)
- PgBouncer non supporta prepared statements → usare sempre query raw con Npgsql
- **SSL Mode=Require** obbligatorio in produzione
- Il DB di sviluppo locale (`gestione_viaggi` su Docker) è una copia esatta dello schema di produzione
- Backup: `Backup_DB/gestione_viaggi_*.backup` (file .backup PostgreSQL custom format)